TRƯỜNG ĐẠI HỌC NHA TRANG

PHÁT TRIỂN PHẦN MỀM HƯỚNG ĐỐI TƯỢNG

CHỦ ĐỀ 1: TỔNG QUAN VỀ TIẾN TRÌNH PHÁT TRIỂN PHẦN MỀM

Nội dung chủ đề 1

- Phần mềm và tiến trình phát triển phần mềm
- Mô hình phát triển phần mềm
- Ngôn ngữ mô hình hợp nhất UML
- Xây dựng đề cương dự án phần mềm

Phần mềm & tiến trình phát triển phần mềm

Phần mềm

- Phần mềm (Software) là một tập hợp các chương trình máy tính, thủ tục (procedures), quy tắc (rules) và tài liệu (documentation) liên quan đến việc vận hành một hệ thống xử lý dữ liệu. Phần mềm không chỉ đơn thuần là mã code, mà còn bao gồm cả kiến trúc thiết kế, tài liệu hướng dẫn sử dụng và các quy trình bảo trì.
- Đặc điểm của phần mềm:
  - ✓**Trừu tượng:** Phần mềm là sản phẩm vô hình, không bị mài mòn theo thời gian như các sản phẩm vật lý.
  - ✓**Phức tạp:** Sự phức tạp của phần mềm thường không đến từ việc sản xuất mà từ quá trình thiết kế và phát triển. Một lỗi nhỏ có thể gây ra những hậu quả lớn.
  - ✓**Dễ thay đổi:** Phần mềm dễ dàng được cập nhật và thay đổi để đáp ứng yêu cầu mới của người dung.

Phần mềm

- Các yêu cầu chất lượng của phần mềm:
  - ✓**Đúng đắn:** Các chức năng của phần mềm phải đầy đủ và chính xác.
  - ✓**Tiện dụng:** Giao diện thân thiện, dễ dung.
  - ✓**Hiệu quả:** Sử dụng tối ưu CPU, bộ nhớ, thiết bị
  - ✓**Tương thích:** Dễ dàng import/export dữ liệu, tương tác với phần mềm khác
  - ✓**Tiến hóa:** Dễ nâng cấp
  - ✓**Bảo mật:** Phân quyền, tránh các xâm nhập bất hợp pháp
  - ✓**An toàn:** có cơ chế sao lưu, phục hồi dữ liệu

Phân loại phần mềm

Phần mềm hệ thống (System Software):

- Là loại phần mềm quản lý và điều khiển các hoạt động của phần cứng máy tính và cung cấp một nền tảng để các ứng dụng khác chạy.

- Ví dụ:
  - Hệ điều hành (Operating Systems - OS): Windows, macOS, Linux, Android, iOS.
  - Trình điều khiển thiết bị (Device Drivers): Phần mềm cho phép hệ điều hành giao tiếp với các thiết bị phần cứng như máy in, card đồ họa, v.v.
  - Tiện ích hệ thống (System Utilities): Các công cụ như chống virus, quản lý tập tin, tối ưu hóa hệ thống.

Phần mềm ứng dụng (Application Software):

- Là loại phần mềm được thiết kế để thực hiện các tác vụ cụ thể cho người dùng.

- Ví dụ:
  - Phần mềm văn phòng (Office Suites): Microsoft Office (Word, Excel, PowerPoint), Google Workspace (Docs, Sheets, Slides).
  - Trình duyệt web (Web Browsers): Chrome, Firefox, Safari, Edge.
  - Phần mềm đồ họa (Graphics Software): Adobe Photoshop, GIMP, Illustrator.
  - Phần mềm giải trí (Entertainment Software): Trò chơi điện tử, trình phát nhạc, trình phát video.
  - Phần mềm quản lý (Management Software): CRM (Customer Relationship Management), ERP (Enterprise Resource Planning).

Phân loại phần mềm

Phần mềm nhúng (Embedded Software):

- Là loại phần mềm được tích hợp vào các thiết bị phần cứng để điều khiển và quản lý các chức năng của thiết bị đó.

- Ví dụ:
  - Phần mềm điều khiển trong ô tô (ECU - Engine Control Unit, ABS - Anti-lock Braking System).
  - Phần mềm trong lò vi sóng, máy giặt, tủ lạnh thông minh.
  - Phần mềm trong các thiết bị IoT (Internet of Things) như cảm biến, thiết bị đeo thông minh.

Phần mềm dựa trên đám mây (Cloud-Based Software):

- Phần mềm được lưu trữ và chạy trên các máy chủ từ xa (trên "đám mây") thay vì trên máy tính cá nhân hoặc máy chủ cục bộ. Người dùng truy cập phần mềm thông qua internet.

- Ví dụ:
  - Phần mềm văn phòng trực tuyến: Google Workspace (Docs, Sheets, Slides), Microsoft Office 365.
  - Phần mềm quản lý quan hệ khách hàng (CRM): Salesforce, HubSpot.
  - Phần mềm lưu trữ và chia sẻ dữ liệu: Google Drive, Dropbox, OneDrive.
  - Phần mềm kế toán: Xero, QuickBooks Online.

Phân loại phần mềm

Phần mềm trí tuệ nhân tạo (AI Software):

- Phần mềm sử dụng các kỹ thuật trí tuệ nhân tạo (AI) như học máy (machine learning), xử lý ngôn ngữ tự nhiên (natural language processing), và thị giác máy tính (computer vision) để thực hiện các tác vụ mà trước đây chỉ có con người mới làm được.

- Ví dụ:
  - Trợ lý ảo: Siri, Google Assistant, Alexa.
  - Phần mềm nhận dạng khuôn mặt.
  - Phần mềm dịch thuật tự động: Google Translate.
  - Phần mềm chatbot: Sử dụng trong dịch vụ khách hàng, hỗ trợ trực tuyến.
  - Phần mềm phân tích dữ liệu: Sử dụng học máy để tìm ra các xu hướng và thông tin chi tiết từ dữ liệu lớn.

Phần mềm mã nguồn mở (Open-Source Software):

- Phần mềm có mã nguồn được công khai, cho phép người dùng xem, sửa đổi và phân phối lại.

- Ví dụ:
  - Hệ điều hành: Linux.
  - Trình duyệt web: Firefox.
  - Phần mềm văn phòng: LibreOffice.
  - Ngôn ngữ lập trình: Python, Java.
  - Cơ sở dữ liệu: MySQL, PostgreSQL.

Phân loại phần mềm

Phần mềm di động (Mobile Software):

- Phần mềm được thiết kế để chạy trên các thiết bị di động như điện thoại thông minh và máy tính bảng.
- Ví dụ:
  - Ứng dụng mạng xã hội: Facebook, Instagram, Twitter.
  - Ứng dụng nhắn tin: WhatsApp, Telegram.
  - Ứng dụng trò chơi.
  - Ứng dụng tiện ích: Bản đồ, thời tiết, lịch.

Phần mềm IoT (Internet of Things Software):

- Phần mềm được sử dụng để kết nối và điều khiển các thiết bị IoT, cho phép chúng giao tiếp và trao đổi dữ liệu với nhau và với các hệ thống khác.
- Ví dụ:
  - Phần mềm điều khiển nhà thông minh.
  - Phần mềm theo dõi sức khỏe trên thiết bị đeo thông minh.
  - Phần mềm quản lý năng lượng trong các tòa nhà thông minh.
  - Phần mềm giám sát môi trường trong nông nghiệp thông minh.

Tiến trình phát triển phần mềm

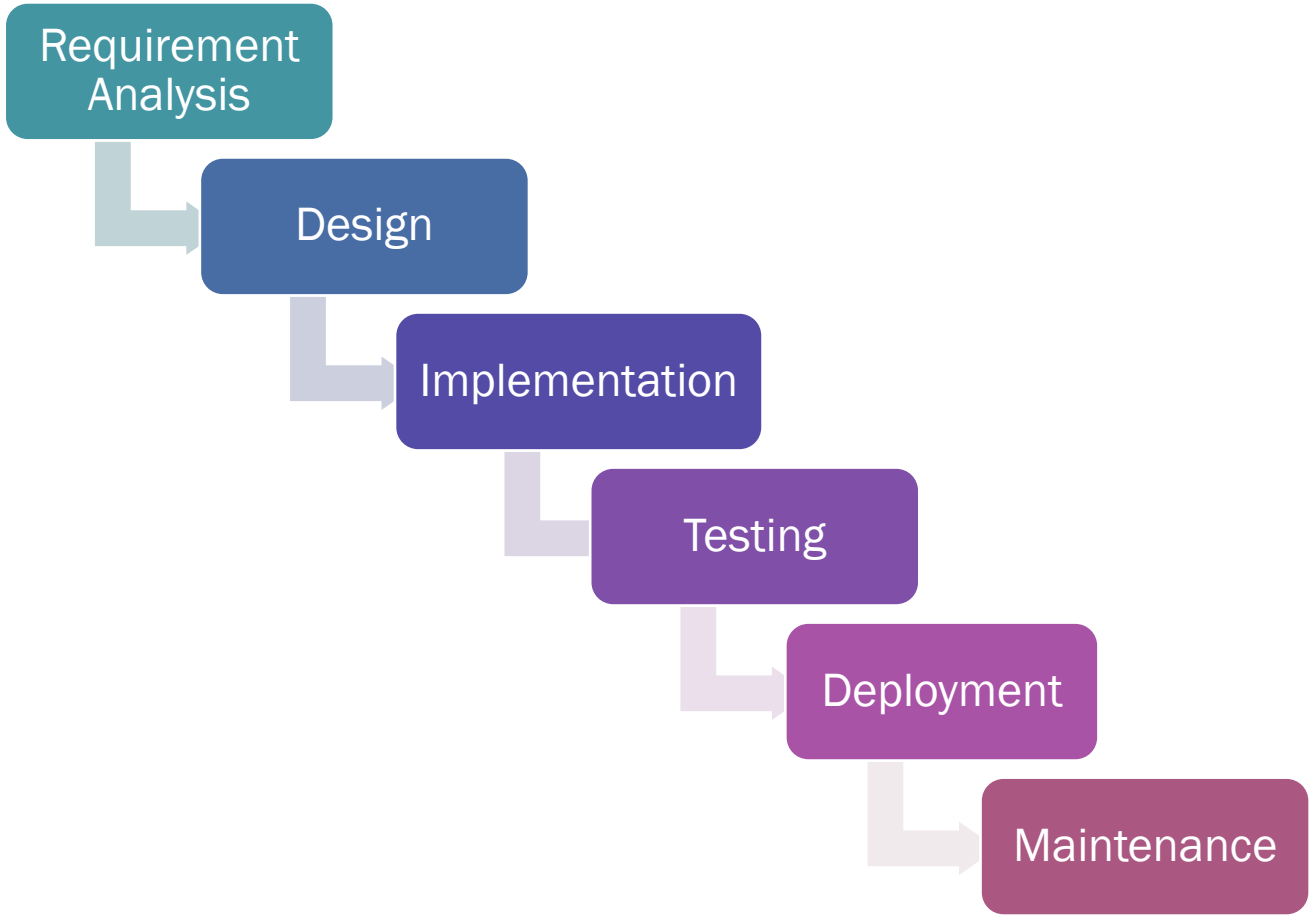
- Tiến trình phát triển phần mềm là gì?
  - Tiến trình phát triển phần mềm (SDLC - Software Development Life Cycle) là một **chuỗi các hoạt động có cấu trúc** nhằm tạo ra, triển khai và bảo trì một sản phẩm phần mềm.
  - SDLC là một quy trình có cấu trúc để sản xuất phần mềm chất lượng cao, chi phí thấp trong thời gian ngắn nhất.
  - SDLC bao gồm một kế hoạch chi tiết với các giai đoạn, mỗi giai đoạn có quy trình và sản phẩm riêng.

Tiến trình phát triển phần mềm

- **Tầm quan trọng của tiến trình phát triển phần mềm**
  - Đảm bảo rằng phần mềm được phát triển một cách có hệ thống, có kế hoạch và có kiểm soát.
  - Giúp giảm thiểu rủi ro, đảm bảo chất lượng, và tăng năng suất trong quá trình phát triển phần mềm.
  - Tạo điều kiện cho việc giao tiếp và phối hợp hiệu quả giữa các thành viên trong nhóm phát triển.
  - Cung cấp một khuôn khổ (framework) để quản lý sự thay đổi và đảm bảo tính bền vững của phần mềm.

Tiến trình phát triển phần mềm

- Các giai đoạn chính trong một tiến trình phát triển phần mềm



Tiến trình phát triển phần mềm

- Các giai đoạn chính trong một tiến trình phát triển phần mềm
  - 1. Phân tích yêu cầu (Requirement Analysis)
    - ✓ Thu thập, phân tích và đặc tả các yêu cầu của người dùng và các bên liên quan (stakeholders).
    - ✓ Sử dụng các kỹ thuật như phỏng vấn (interviews), khảo sát (surveys), phân tích tài liệu (document analysis), và xây dựng mô hình (modeling).
    - ✓ Kết quả: Tài liệu đặc tả yêu cầu (requirements specification document), mô hình use case, mô hình nghiệp vụ.

Tiến trình phát triển phần mềm

- Các giai đoạn chính trong một tiến trình phát triển phần mềm
  - 2. Thiết kế (Design)
    - ✓ Xây dựng kiến trúc và thiết kế chi tiết của hệ thống phần mềm dựa trên các yêu cầu đã được xác định.
    - ✓ Sử dụng các kỹ thuật như thiết kế hướng đối tượng (object-oriented design), thiết kế kiến trúc (architectural design), và thiết kế giao diện người dùng (user interface design).
    - ✓ Kết quả: Tài liệu thiết kế (design document), mô hình lớp (class diagram), mô hình cơ sở dữ liệu (database schema), thiết kế giao diện người dùng (UI design).

Tiến trình phát triển phần mềm

- Các giai đoạn chính trong một tiến trình phát triển phần mềm
  - 3. Lập trình (Implementation/Coding/Programming)
    - ✓ Chuyển đổi thiết kế thành mã nguồn (source code) bằng cách sử dụng một hoặc nhiều ngôn ngữ lập trình.
    - ✓ Tuân thủ các tiêu chuẩn lập trình (coding standards) và thực hiện kiểm tra mã (code reviews) để đảm bảo chất lượng mã.
    - ✓ Kết quả: Mã nguồn (source code), thư viện (libraries), thành phần (components).

Tiến trình phát triển phần mềm

- Các giai đoạn chính trong một tiến trình phát triển phần mềm
  - 4. Kiểm thử (Testing)
    - ✓ Xác minh (verification) và xác nhận (validation) rằng phần mềm hoạt động đúng như mong đợi và đáp ứng các yêu cầu chất lượng.
    - ✓ Sử dụng các kỹ thuật kiểm thử như kiểm thử đơn vị (unit testing), kiểm thử tích hợp (integration testing), kiểm thử hệ thống (system testing), và kiểm thử chấp nhận (acceptance testing).
    - ✓ Kết quả: Báo cáo kiểm thử (test reports), danh sách lỗi (bug lists), bản sửa lỗi (bug fixes).

Tiến trình phát triển phần mềm

- Các giai đoạn chính trong một tiến trình phát triển phần mềm
  - 5. Triển khai (Deployment)
    - ✓ Cài đặt và cấu hình phần mềm trên môi trường sản xuất (production environment).
    - ✓ Đảm bảo rằng phần mềm hoạt động ổn định và có thể truy cập được bởi người dùng.
    - ✓ Thực hiện các hoạt động như di chuyển dữ liệu (data migration), cấu hình máy chủ (server configuration), và đào tạo người dùng (user training).

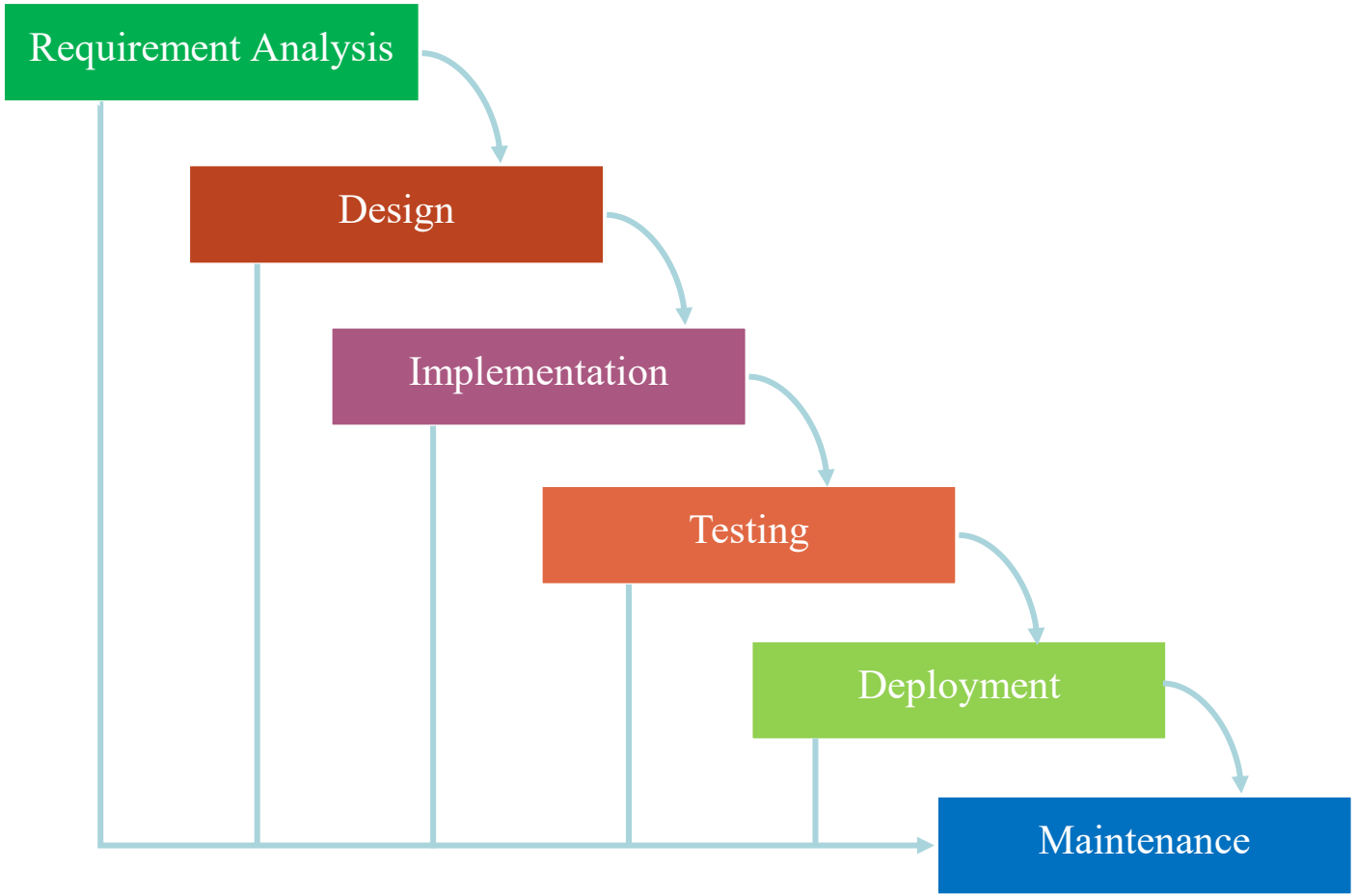
Tiến trình phát triển phần mềm

- Các giai đoạn chính trong một tiến trình phát triển phần mềm
  - 6. Bảo trì (Maintenance)
    - ✓ Sửa chữa lỗi (bug fixing), cải tiến hiệu suất (performance optimization), và bổ sung các tính năng mới (feature enhancements) cho phần mềm sau khi đã được triển khai.
    - ✓ Thực hiện các hoạt động như theo dõi hiệu suất (performance monitoring), phân tích nhật ký (log analysis), và quản lý thay đổi (change management).

Mô hình phát triển phần mềm

Tiến trình phát triển phần mềm

- Mô hình thác nước (Waterfall model)
  - Đây là mô hình truyền thống và cổ điển nhất. Các giai đoạn của tiến trình phát triển phần mềm được thực hiện một cách tuần tự và tuyến tính. Giai đoạn sau chỉ được bắt đầu khi giai đoạn trước đã hoàn thành hoàn toàn.

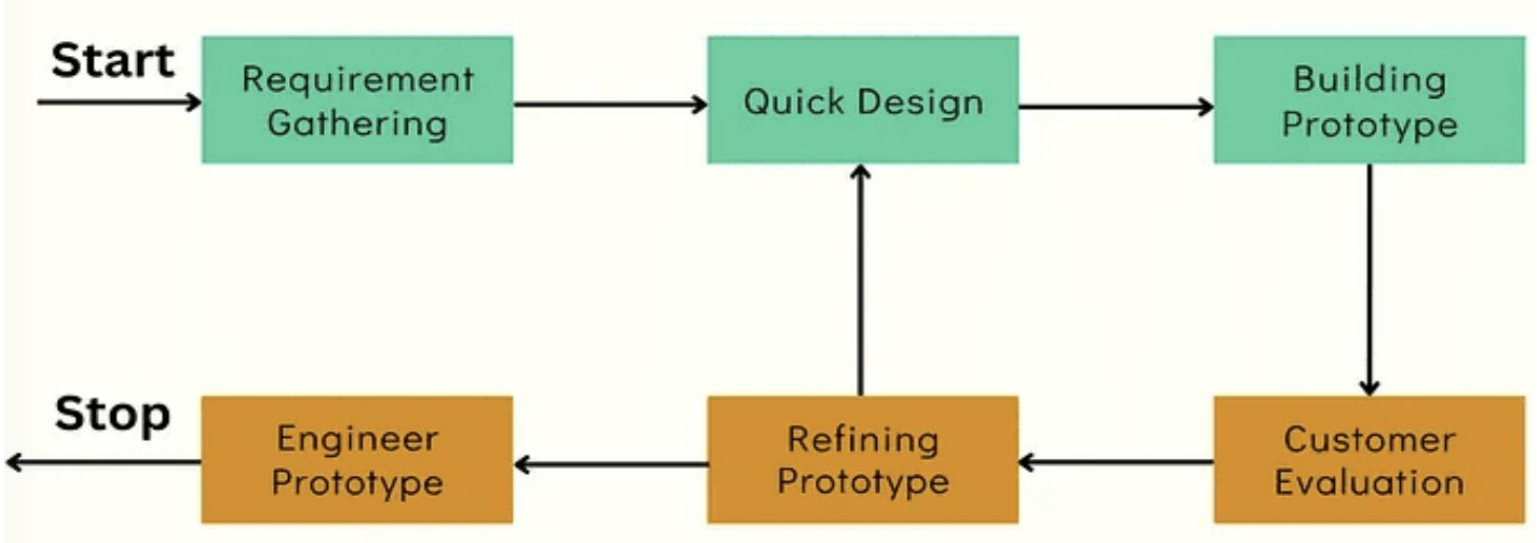


Tiến trình phát triển phần mềm

- Mô hình thác nước (Waterfall model)
  - **Ưu điểm:**
    - ✓ Đơn giản và dễ quản lý: Do tính chất tuần tự, mỗi giai đoạn có các mục tiêu và tài liệu rõ ràng.
    - ✓ Phù hợp với dự án nhỏ: Khi yêu cầu được xác định đầy đủ và không thay đổi, mô hình này rất hiệu quả.
    - ✓ Dễ kiểm soát: Tiến độ dự án dễ dàng được theo dõi.
  - **Nhược điểm:**
    - ✓ Thiếu linh hoạt: Rất khó và tốn kém để quay lại các giai đoạn trước để sửa đổi.
    - ✓ Rủi ro cao: Lỗi chỉ được phát hiện ở giai đoạn cuối, làm tăng chi phí sửa chữa.
    - ✓ Không phù hợp với yêu cầu thay đổi: Trong môi trường kinh doanh năng động, các yêu cầu thường xuyên thay đổi, khiến mô hình này kém hiệu quả.
  - **Khi nào nên sử dụng:** Khi các yêu cầu của dự án đã được **xác định rõ ràng, ổn định** và không có khả năng thay đổi. Ví dụ: dự án phát triển phần mềm nhúng cho các thiết bị phần cứng cụ thể.

Tiến trình phát triển phần mềm

- Mô hình nguyên mẫu (Prototyping model)
  - Mô hình này tập trung vào việc tạo ra một phiên bản thử nghiệm (prototype) của sản phẩm để thu thập phản hồi từ người dùng. Sau đó, prototype này được tinh chỉnh và phát triển thành sản phẩm cuối cùng. Quy trình lặp lại cho đến khi sản phẩm đáp ứng yêu cầu.

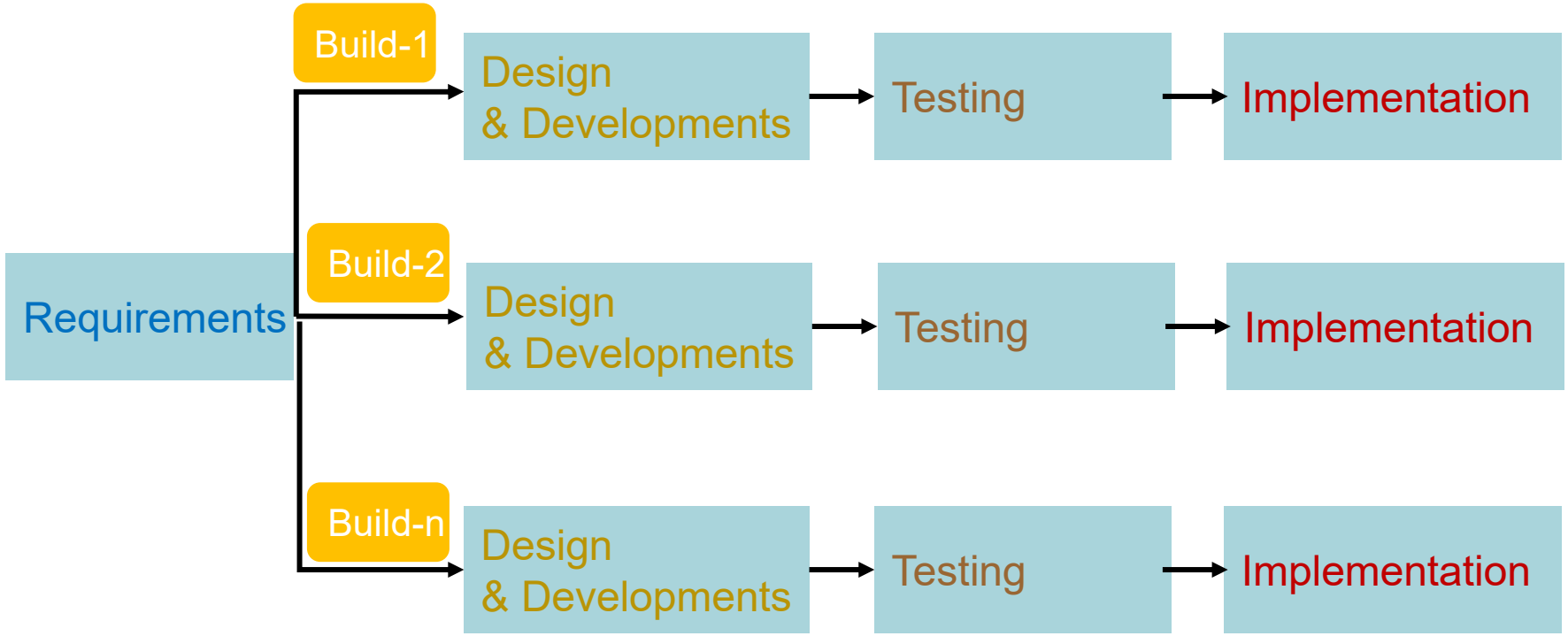


Tiến trình phát triển phần mềm

- Mô hình nguyên mẫu (Prototyping model)
  - **Ưu điểm:**
    - ✓ Giảm thiểu rủi ro yêu cầu: Người dùng có thể trải nghiệm sản phẩm sớm và cung cấp phản hồi, giúp đội ngũ phát triển hiểu rõ yêu cầu hơn.
    - ✓ Cải thiện giao tiếp: Cả đội ngũ phát triển và khách hàng đều có thể "nhìn thấy" sản phẩm, tránh hiểu lầm.
    - ✓ Sản phẩm có chất lượng cao hơn: Phản hồi lặp đi lặp lại giúp sản phẩm cuối cùng sát với mong muốn của người dùng
  - **Nhược điểm:**
    - ✓ Có thể tốn kém: Việc tạo ra các prototype có thể tốn thời gian và nguồn lực.
    - ✓ Mất kiểm soát: Nếu không được quản lý chặt chẽ, chu trình tạo prototype có thể kéo dài vô tận.
  - **Khi nào nên sử dụng:** Khi yêu cầu của người dùng chưa rõ ràng, đặc biệt là về giao diện và trải nghiệm người dùng.

Tiến trình phát triển phần mềm

- Mô hình gia tăng (Incremental model)
  - Thay vì phát triển toàn bộ sản phẩm cùng một lúc, mô hình này chia nhỏ sản phẩm thành các **phần có thể hoạt động được (increments)**. Mỗi phần được phát triển và triển khai theo một chu trình Thác nước nhỏ. Các phần này được tích hợp với nhau cho đến khi tạo thành sản phẩm hoàn chỉnh.

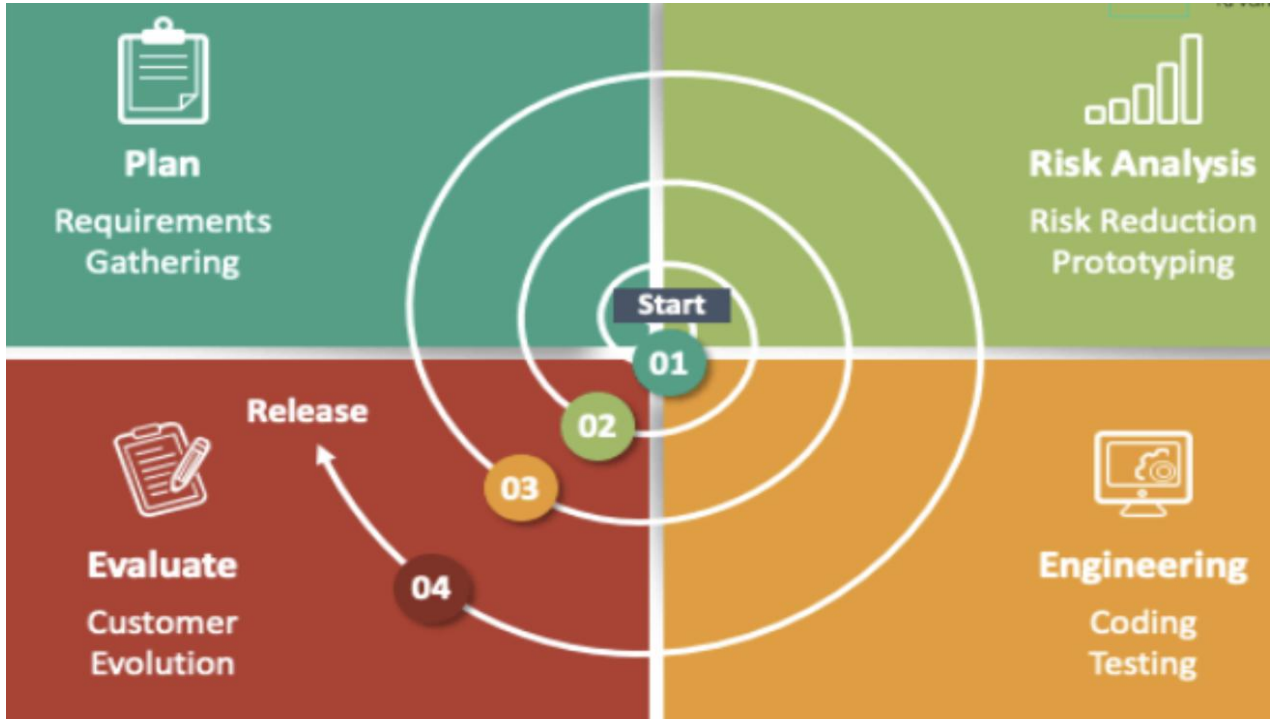


Tiến trình phát triển phần mềm

- Mô hình gia tăng (Incremental model)
  - **Ưu điểm:**
    - ✓ Triển khai sản phẩm sớm: Phiên bản đầu tiên có thể được triển khai với các chức năng cơ bản, sau đó được bổ sung dần.
    - ✓ Giảm rủi ro: Rủi ro được phân tán và dễ quản lý hơn vì chỉ tập trung vào một phần nhỏ của sản phẩm tại một thời điểm.
    - ✓ Phản hồi sớm: Người dùng có thể sử dụng sản phẩm từ sớm và đưa ra phản hồi.
  - **Nhược điểm:**
    - ✓ Yêu cầu thiết kế tổng thể tốt: Nếu kiến trúc ban đầu không vững chắc, việc tích hợp các phần sau này có thể gặp khó khăn.
    - ✓ Chi phí có thể tăng: Chi phí cho mỗi chu trình nhỏ có thể cộng dồn lại.
  - **Khi nào nên sử dụng:** Khi các chức năng của sản phẩm có thể được chia thành các phần nhỏ, độc lập và có thể ưu tiên. Ví dụ: một ứng dụng web, với các chức năng đăng nhập, quản lý hồ sơ, và thanh toán được phát triển theo các giai đoạn riêng biệt.

Tiến trình phát triển phần mềm

- Mô hình Xoắn ốc (Spiral model)
  - Mô hình này kết hợp tính lặp lại của mô hình prototype và tính kiểm soát của mô hình thác nước. Mỗi vòng xoắn trong mô hình là một chu trình phát triển. Mô hình tập trung vào việc **quản lý rủi ro** và được thực hiện qua bốn pha trong mỗi vòng lặp: Lập kế hoạch, Phân tích rủi ro, Kỹ thuật (Phát triển), và Đánh giá.



Tiến trình phát triển phần mềm

- Mô hình xoắn ốc (Spiral model)
  - **Ưu điểm:**
    - ✓ Quản lý rủi ro: Giảm thiểu rủi ro một cách hệ thống trong mỗi vòng lặp.
    - ✓ Linh hoạt cao: Cho phép thay đổi yêu cầu và bổ sung tính năng ở mỗi vòng lặp.
    - ✓ Phù hợp với các dự án lớn: Đặc biệt hiệu quả với các dự án phức tạp, quy mô lớn, nơi rủi ro là yếu tố then chốt.
  - **Nhược điểm:**
    - ✓ Phức tạp và tốn kém: Đòi hỏi đội ngũ có kinh nghiệm cao và chi phí lớn.
    - ✓ Không phù hợp với dự án nhỏ: Sự phức tạp của mô hình có thể không cần thiết cho các dự án đơn giản.
  - **Khi nào nên sử dụng:** Khi phát triển các hệ thống lớn, phức tạp và có tính chất rủi ro cao, ví dụ: hệ thống an ninh, hệ thống hàng không.

Tiến trình phát triển phần mềm

- Mô hình Agile (Spiral model)
  - Agile là một **phương pháp phát triển phần mềm linh hoạt**, nhấn mạnh vào việc chia nhỏ dự án thành các giai đoạn ngắn (iteration/sprint), có sự tham gia thường xuyên của khách hàng và khả năng thích ứng nhanh với sự thay đổi.



Tiến trình phát triển phần mềm

- Mô hình Agile
  - **Ưu điểm:**
    - ✓ Thích ứng cao: Dễ dàng thay đổi yêu cầu và ưu tiên trong quá trình phát triển.
    - ✓ Sản phẩm được triển khai nhanh chóng: Liên tục đưa ra các phiên bản nhỏ, có chức năng hoàn chỉnh.
    - ✓ Tăng sự hài lòng của khách hàng: Khách hàng tham gia trực tiếp, thấy sản phẩm phát triển dần.
  - **Nhược điểm:**
    - ✓ Yêu cầu đội ngũ có kinh nghiệm: Cần sự hợp tác chặt chẽ, kỷ luật và tự quản lý cao.
    - ✓ Khó áp dụng cho dự án lớn: Nếu không được quản lý tốt, dự án có thể bị phân tán.
    - ✓ Tài liệu có thể không đầy đủ: Agile tập trung vào sản phẩm hoạt động hơn là tài liệu.
  - **Khi nào nên sử dụng:** Khi yêu cầu dự án thay đổi thường xuyên, cần ra sản phẩm nhanh, dự án có sự tham gia tích cực của khách hàng.

Ngôn ngữ mô hình hợp nhất UML



Giới thiệu UML

- UML (Unified Modeling Language) là một ngôn ngữ mô hình hóa trực quan tiêu chuẩn được sử dụng trong kỹ thuật phần mềm để đặc tả, trực quan hóa, xây dựng và tài liệu hóa các thành phần của hệ thống phần mềm..
- UML không phải là ngôn ngữ lập trình, mà là một **chuẩn mô hình hóa** do OMG (Object Management Group) phát triển, dùng để biểu diễn trực quan các thiết kế phần mềm hướng đối tượng.
- Mục tiêu chính của UML:
  - Giúp các nhà phát triển, phân tích viên, khách hàng hiểu rõ hơn về hệ thống một cách trực quan.
  - Tạo cầu nối giữa phân tích, thiết kế và lập trình.
  - Chuẩn hóa cách mô tả hệ thống để tránh mơ hồ.

Đặc trưng của UML

- Tính trực quan (Visual): UML sử dụng các biểu đồ (diagrams) để biểu diễn các khía cạnh khác nhau của hệ thống phần mềm, giúp cho việc hiểu và giao tiếp trở nên dễ dàng hơn.
- Tính tiêu chuẩn (Standardized): UML là một tiêu chuẩn được công nhận rộng rãi, giúp cho việc trao đổi thông tin và tài liệu giữa các nhóm phát triển và tổ chức khác nhau trở nên dễ dàng hơn.
- Tính linh hoạt (Flexible): UML có thể được sử dụng để mô hình hóa các hệ thống phần mềm thuộc nhiều loại khác nhau, từ các ứng dụng nhỏ đến các hệ thống lớn và phức tạp.
- Tính hỗ trợ công cụ (Tool-Supported): Có rất nhiều công cụ phần mềm hỗ trợ UML, giúp cho việc tạo, chỉnh sửa và quản lý các mô hình UML trở nên dễ dàng và hiệu quả hơn.

UML và các giai đoạn của tiến trình phần mềm

UML hỗ trợ trong hầu hết các giai đoạn của Software Development Life Cycle (SDLC):

1) Giai đoạn phân tích yêu cầu:
   - Sử dụng UML để mô tả yêu cầu chức năng, nghiệp vụ.
   - Biểu đồ thường dùng: *Use Case Diagram, Activity Diagram, Sequence Diagram*
2) Giai đoạn thiết kế hệ thống:
   - Xây dựng kiến trúc tổng thể và chi tiết cho hệ thống.
   - Biểu đồ thường dùng: *Class Diagram, Component Diagram, Deployment Diagram*
3) Giai đoạn cài đặt:
   - UML hỗ trợ lập trình viên hiểu rõ cấu trúc và quan hệ giữa các module.
   - UML có thể được sử dụng để tạo mã nguồn tự động (code generation) từ các mô hình UML.
   - Biểu đồ: *Class Diagram, Object Diagram, Package Diagram.*
5) Giai đoạn kiểm thử:
   - Dựa vào kịch bản trong *Use Case Diagram, Sequence Diagram* để viết test case.
6) Giai đoạn triển khai và bảo trì:
   - UML mô tả cấu trúc triển khai (*Deployment Diagram*), dễ dàng bảo trì và mở rộng.

Các loại biểu đồ trong UML

UML cung cấp 14 loại biểu đồ khác nhau, được chia thành hai nhóm chính:

Biểu đồ Cấu trúc (Structure Diagrams): Mô tả cấu trúc tĩnh của hệ thống

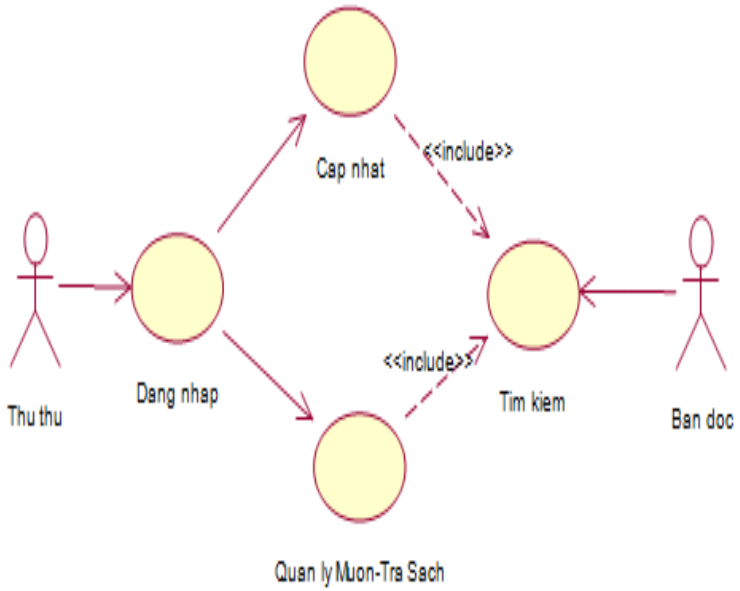
- Biểu đồ Lớp (Class Diagram)
- Biểu đồ Đối tượng (Object Diagram)
- Biểu đồ Gói (Package Diagram)
- Biểu đồ Thành phần (Component Diagram)
- Biểu đồ Triển khai (Deployment Diagram)
- Biểu đồ Cấu trúc Tổ hợp (Composite Structure Diagram)
- Biểu đồ Hồ sơ (Profile Diagram)

Biểu đồ Hành vi (Behavior Diagrams): Mô tả hành vi động của hệ thống

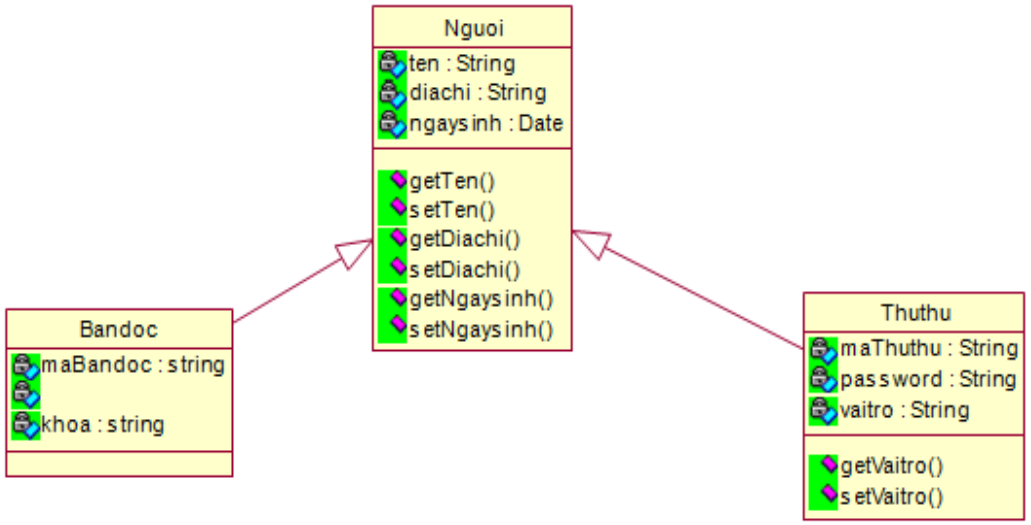
- Biểu đồ Use Case (Use Case Diagram)
- Biểu đồ Hoạt động (Activity Diagram)
- Biểu đồ Trạng thái (State Diagram)
- Biểu đồ Tuần tự (Sequence Diagram)
- Biểu đồ Giao tiếp (Communication Diagram)
- Biểu đồ Thời gian (Timing Diagram)
- Biểu đồ Tương tác Tổng quan (Interaction Overview Diagram)

Các loại biểu đồ trong UML (Ví dụ)

- Biểu đồ Use case:

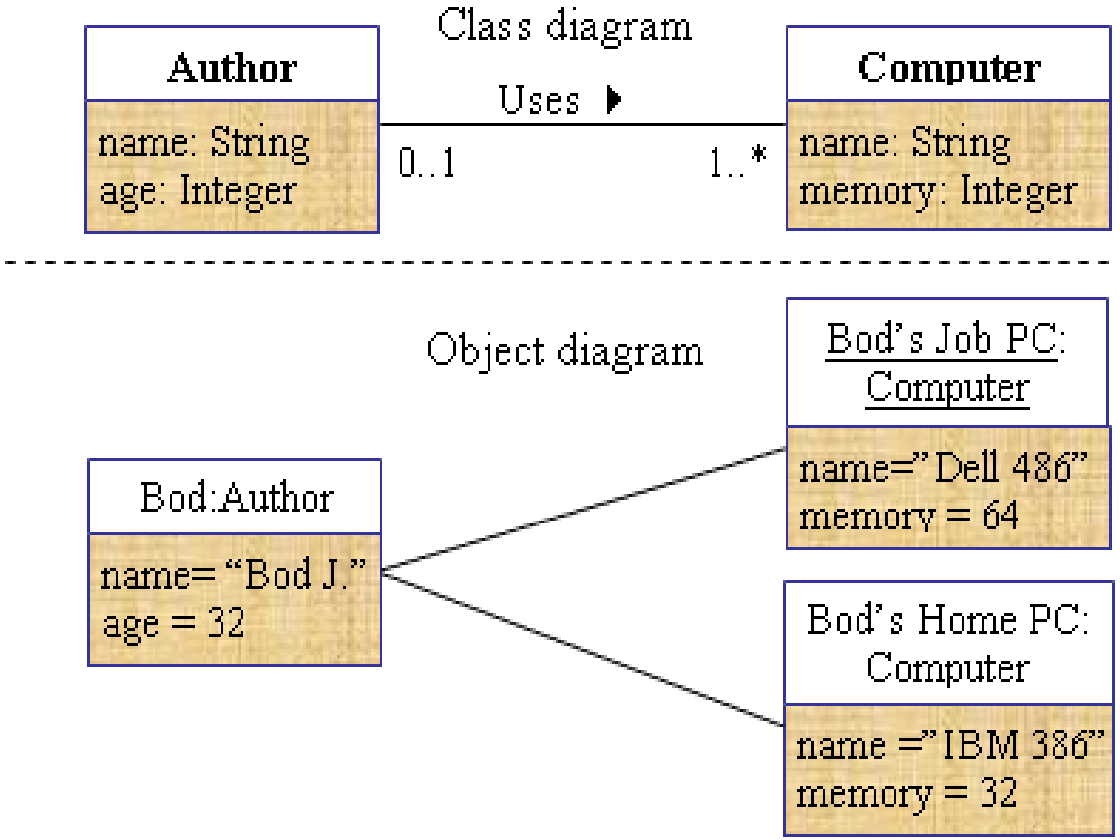


- Biểu đồ lớp:

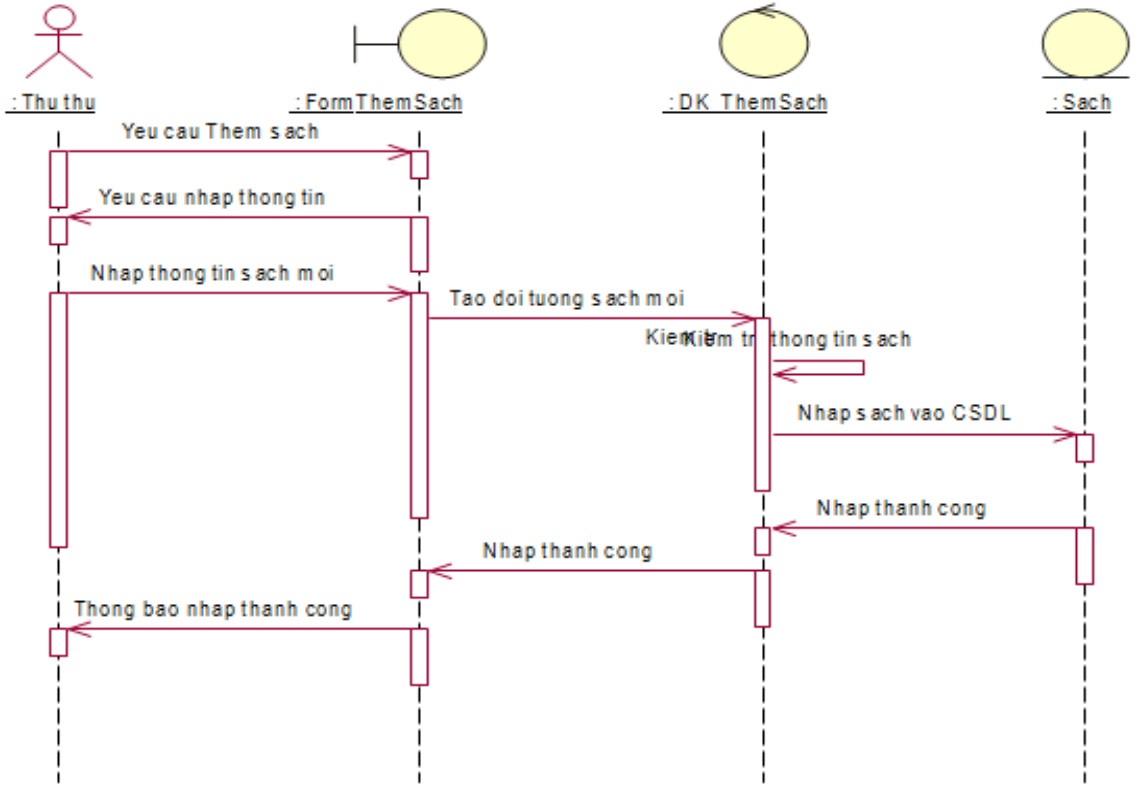


Các loại biểu đồ trong UML (Ví dụ)

- Biểu đồ đối tượng:

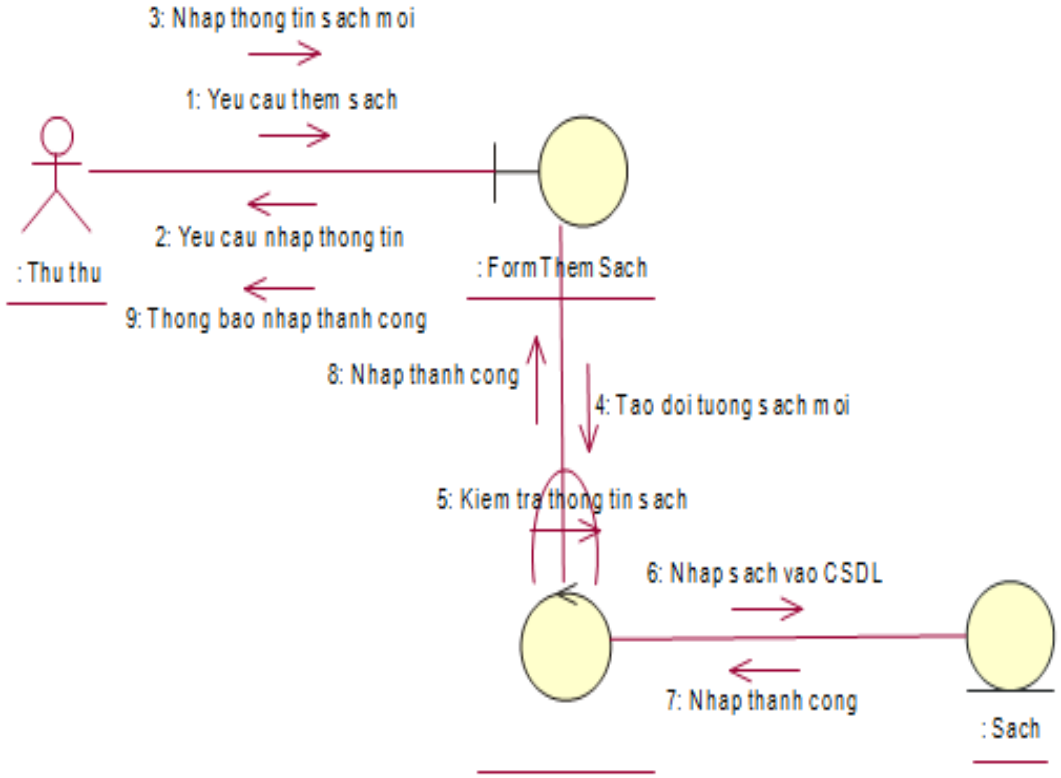


- Biểu đồ trình tự:

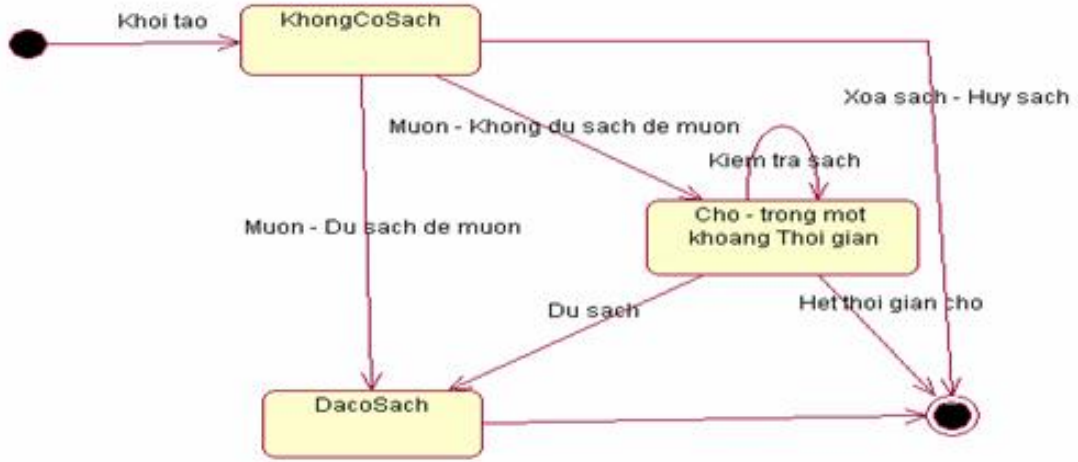


Các loại biểu đồ trong UML (Ví dụ)

- Biểu đồ cộng tác:

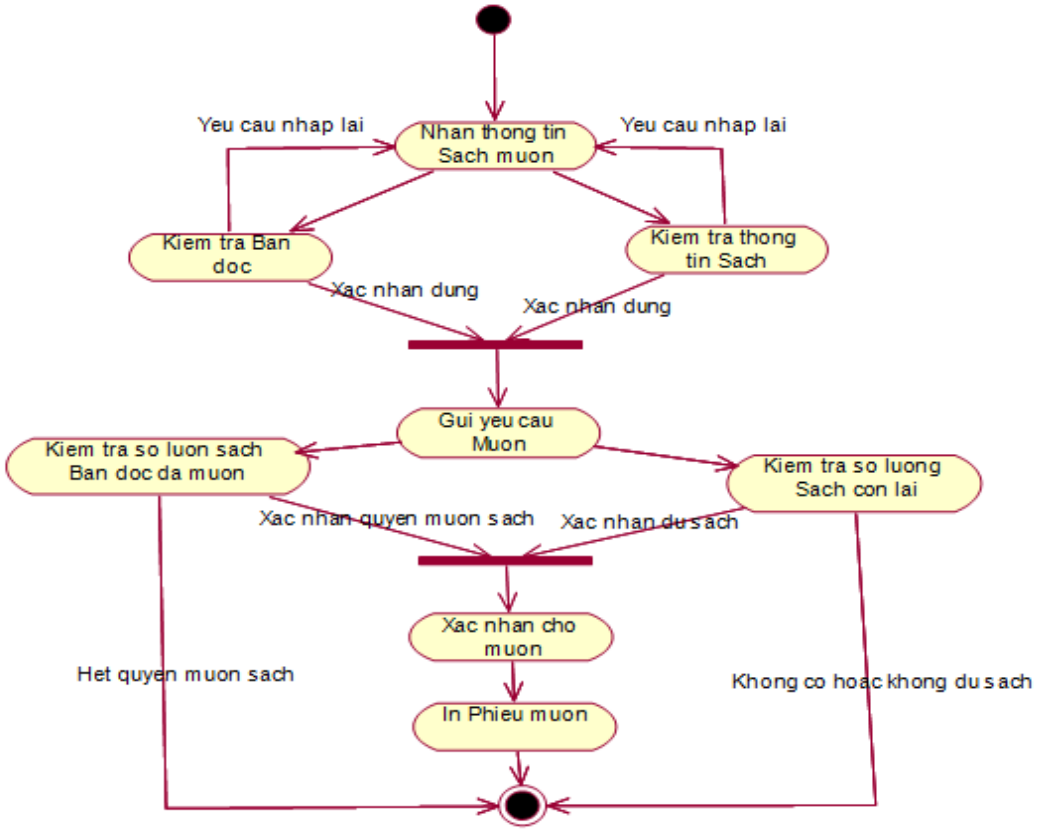


- Biểu đồ trạng thái:

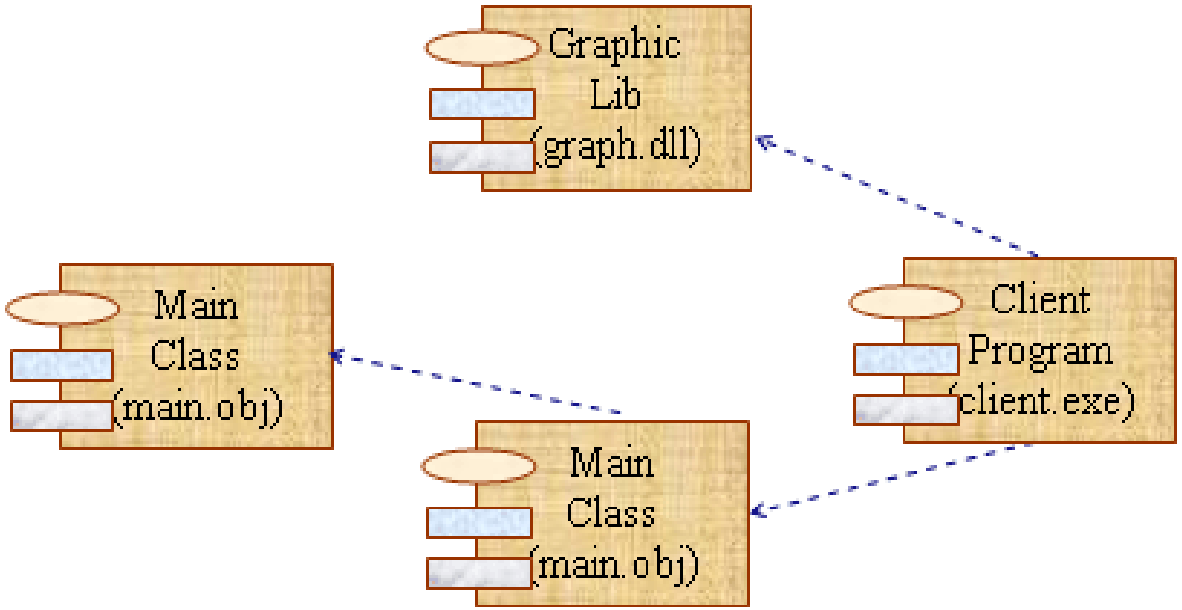


Các loại biểu đồ trong UML (Ví dụ)

- Biểu đồ hoạt động:

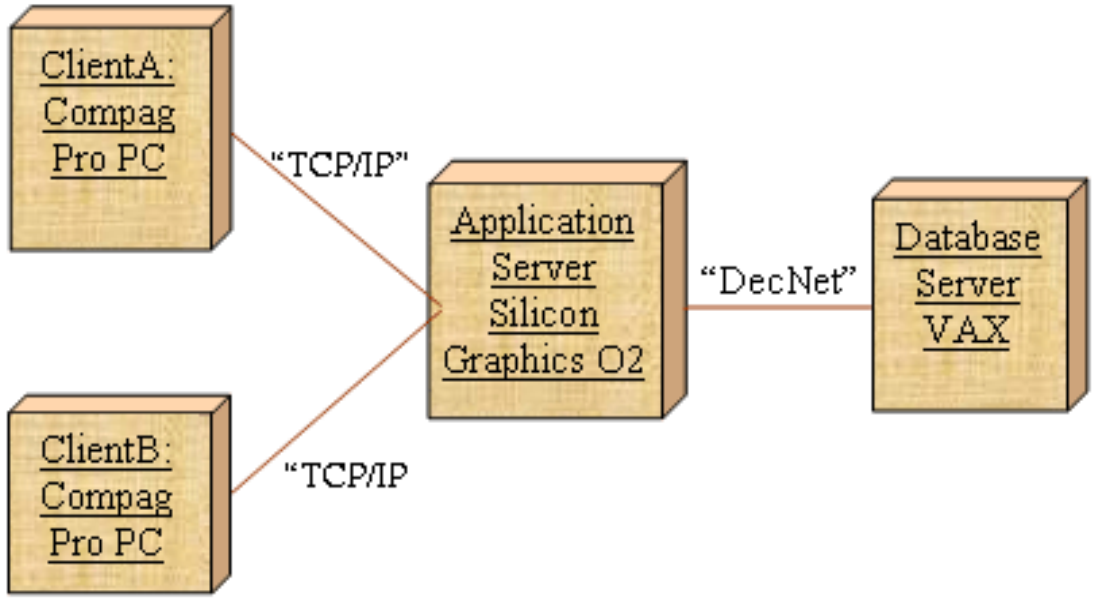


- Biểu đồ thành phần:



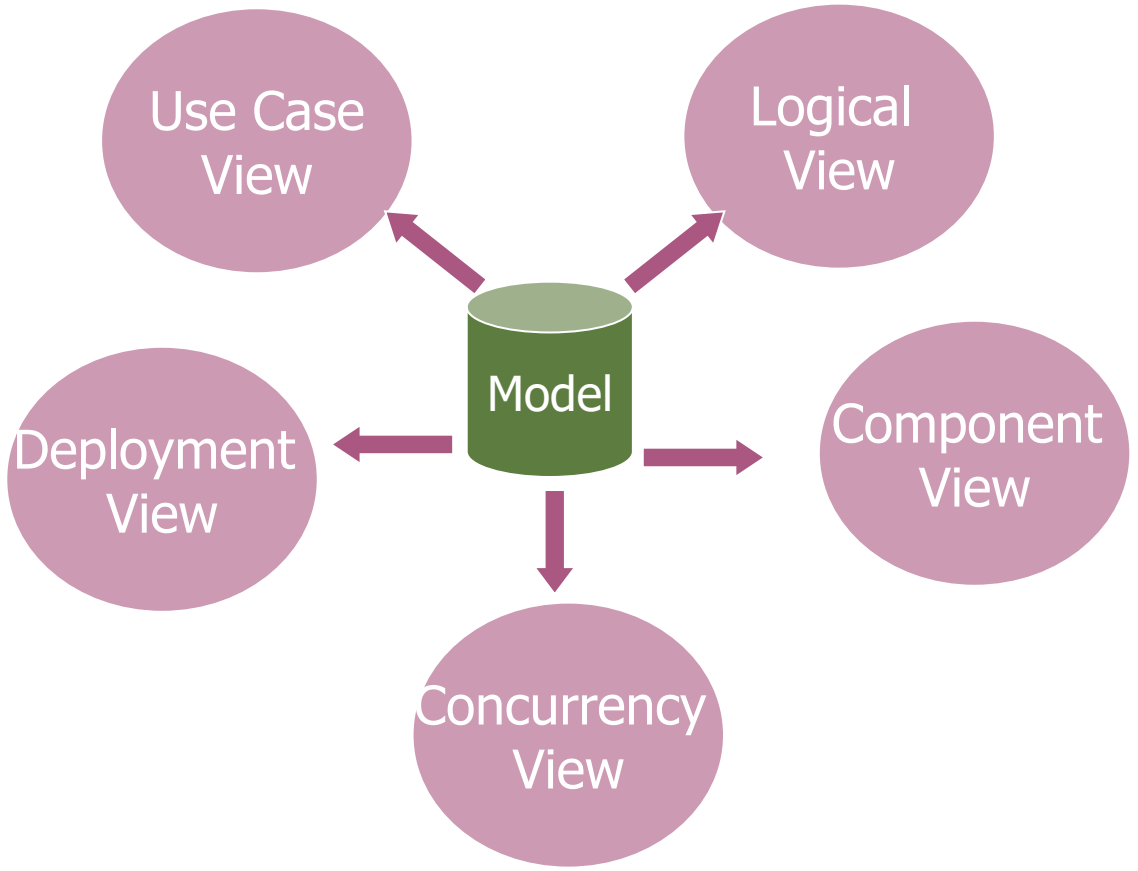
Các loại biểu đồ trong UML (Ví dụ)

- Biểu đồ triển khai:



Hướng nhìn trong UML

- Khái niệm:
  - Trong UML, một "hướng nhìn" (view) là một cách để thể hiện một phần của hệ thống từ một góc độ cụ thể. Nó là một sự trừu tượng hóa của hệ thống, tập trung vào một tập hợp các khía cạnh liên quan và bỏ qua các chi tiết không liên quan.
  - Hướng nhìn giúp đơn giản hóa việc hiểu và quản lý các hệ thống phần mềm phức tạp bằng cách chia chúng thành các phần nhỏ hơn, dễ quản lý hơn.
  - Một hệ thống UML có thể có nhiều hướng nhìn khác nhau, mỗi hướng nhìn tập trung vào một khía cạnh khác nhau của hệ thống.



Hướng nhìn trong UML

- Mục đích của hướng nhìn:
  - Đơn giản hóa: Giảm độ phức tạp bằng cách chỉ hiển thị các thông tin liên quan đến một khía cạnh cụ thể.
  - Tập trung: Cho phép các bên liên quan (stakeholders) tập trung vào các khía cạnh quan trọng nhất đối với họ.
  - Giao tiếp: Tạo ra một ngôn ngữ chung để các bên liên quan có thể giao tiếp và hiểu nhau một cách hiệu quả.
  - Quản lý: Giúp quản lý các hệ thống phức tạp bằng cách chia chúng thành các phần nhỏ hơn, dễ quản lý hơn.

Các loại hướng nhìn trong UML

UML không định nghĩa một tập hợp các hướng nhìn cố định, nhưng có một số hướng nhìn phổ biến thường được sử dụng trong thực tế:

Hướng nhìn Use Case (Use Case View)

- Tập trung vào các chức năng của hệ thống từ quan điểm của người dùng.
- Mô tả các tương tác giữa người dùng và hệ thống để đạt được các mục tiêu cụ thể.
- Sử dụng biểu đồ Use Case (Use Case Diagram) để biểu diễn.

Hướng nhìn Logic (Logical View)

- Tập trung vào cấu trúc tĩnh và hành vi động của hệ thống.
- Mô tả các lớp, đối tượng, thuộc tính, phương thức và mối quan hệ giữa chúng.
- Sử dụng biểu đồ Lớp (Class Diagram), biểu đồ Đối tượng (Object Diagram), biểu đồ Tuần tự (Sequence Diagram), biểu đồ Giao tiếp (Communication Diagram), và biểu đồ Trạng thái (State Diagram) để biểu diễn.

Các loại hướng nhìn trong UML

Hướng nhìn Tiến trình (Process View)

- Tập trung vào các tiến trình và luồng điều khiển trong hệ thống.
- Mô tả cách các tiến trình tương tác với nhau và cách dữ liệu được truyền giữa chúng.
- Sử dụng biểu đồ Hoạt động (Activity Diagram) để biểu diễn.

Hướng nhìn Triển khai (Deployment View)

- Tập trung vào cấu trúc vật lý của hệ thống.
- Mô tả cách các thành phần phần mềm được triển khai trên các nút phần cứng.
- Sử dụng biểu đồ Thành phần (Component Diagram) và biểu đồ Triển khai (Deployment Diagram) để biểu diễn.

Hướng nhìn Dữ liệu (Data View)

- Tập trung vào cấu trúc và quản lý dữ liệu của hệ thống.
- Mô tả các bảng, cột, khóa và mối quan hệ trong cơ sở dữ liệu.
- Sử dụng biểu đồ Lớp (Class Diagram) (để mô hình hóa các đối tượng dữ liệu) và biểu đồ ER (Entity-Relationship Diagram) (nếu có) để biểu diễn.

Hướng nhìn & Biểu đồ UML

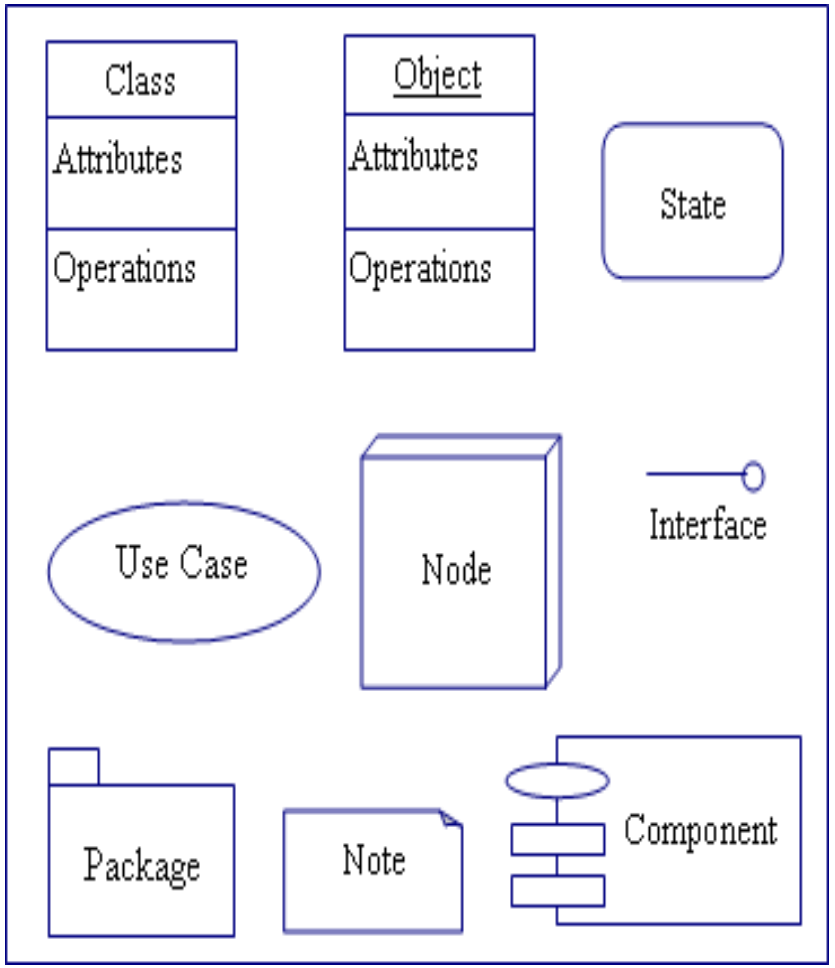
- Hướng nhìn là một tập hợp các diagram: Một hướng nhìn thường bao gồm một hoặc nhiều diagram UML, mỗi diagram tập trung vào một khía cạnh cụ thể của hướng nhìn đó.
- Diagram có thể thuộc nhiều hướng nhìn: Một diagram có thể được sử dụng trong nhiều hướng nhìn khác nhau, tùy thuộc vào mục đích sử dụng.
- Hướng nhìn cung cấp ngữ cảnh cho diagram: Hướng nhìn cung cấp một ngữ cảnh rõ ràng cho các diagram, giúp người xem hiểu rõ hơn về ý nghĩa và mục đích của chúng.
- Diagram hỗ trợ hướng nhìn: Các diagram cung cấp các chi tiết cụ thể để hỗ trợ và minh họa cho các khái niệm và mối quan hệ được mô tả trong hướng nhìn.

Hướng nhìn & Biểu đồ UML

- Ví dụ: Giả sử chúng ta đang phát triển một hệ thống quản lý thư viện (library management system)
  - Hướng nhìn Use Case: Biểu đồ Use Case sẽ mô tả các chức năng chính của hệ thống, chẳng hạn như "Đăng ký thành viên," "Mượn sách," "Trả sách," "Tìm kiếm sách," và "Quản lý sách."
  - Hướng nhìn Logic:
    - ✓ Biểu đồ Lớp sẽ mô tả các lớp chính trong hệ thống, chẳng hạn như "Thành viên," "Sách," "Phiếu mượn," và "Nhân viên."
    - ✓ Biểu đồ Tuần tự sẽ mô tả các tương tác giữa các đối tượng này khi thực hiện các chức năng như "Mượn sách" hoặc "Trả sách."
  - Hướng nhìn Dữ liệu: Biểu đồ Lớp (hoặc biểu đồ ER) sẽ mô tả cấu trúc của cơ sở dữ liệu, bao gồm các bảng, cột và mối quan hệ giữa chúng.

Các thành phần cơ bản của UML

- Phần tử mô hình (Model Elements): Các khái niệm cơ bản trong UML, được sử dụng để xây dựng các mô hình.
  - Lớp (Class): Biểu diễn một tập hợp các đối tượng có chung thuộc tính, phương thức và quan hệ.
  - Đối tượng (Object): Một thể hiện cụ thể của một lớp.
  - Giao diện (Interface): Một tập hợp các phương thức mà một lớp hoặc thành phần phải thực hiện.
  - Use Case: Mô tả một tương tác giữa người dùng và hệ thống để đạt được một mục tiêu cụ thể.
  - Thành phần (Component): Một phần tử có thể thay thế được của một hệ thống, đóng gói một số chức năng và cung cấp các giao diện để giao tiếp với các thành phần khác.
  - Nút (Node): Một phần tử vật lý trong hệ thống, chẳng hạn như một máy chủ, máy tính cá nhân hoặc thiết bị di động.
  - Chú thích (Annotations): Các ghi chú hoặc giải thích được thêm vào mô hình để cung cấp thêm thông tin.



Các thành phần cơ bản của UML

- Quan hệ (Relationships): Các kết nối giữa các phần tử mô hình.
  - Kết hợp (Association): Biểu diễn một mối quan hệ giữa hai lớp.
  - Tổng hợp (Aggregation): Một dạng đặc biệt của kết hợp, biểu diễn mối quan hệ "có một" (has-a).
  - Hợp thành (Composition): Một dạng đặc biệt của tổng hợp, biểu diễn mối quan hệ "là một phần của" (is-part-of).
  - Kế thừa (Inheritance): Biểu diễn mối quan hệ "là một loại" (is-a) giữa một lớp cha (superclass) và một lớp con (subclass).
  - Thực hiện (Realization): Biểu diễn mối quan hệ giữa một lớp và một giao diện mà nó thực hiện.
  - Phụ thuộc (Dependency): Biểu diễn một mối quan hệ yếu giữa hai phần tử, trong đó một phần tử phụ thuộc vào phần tử kia.

